CO3-AT1 EXPREIMENT

Analyze the performance of Merge Sort by implementing the algorithm on datasets with different input sizes and recording execution time for each case. Plot the results using graphs to visualize time complexity trends. Interpret how the divide and conquer strategy ensures consistent O(n log n) performance regardless of input distribution. Justify why Merge Sort is preferred for stable sorting applications involving large datasets.

Merge Sort – Performance Analysis

1. Introduction

Merge Sort is a sorting algorithm based on the **Divide and Conquer** technique. It divides the input array into smaller subarrays, sorts them separately, and finally merges them to obtain the sorted array.

The main objective of this experiment is to analyze the performance of Merge Sort for different input sizes and understand its **O(n log n)** time complexity.

□genui□{"learning_viz":{"type_id":"MERGE_SORT"}}□

---

2. Working of Merge Sort

Merge Sort works in three main steps:

1. **Divide** – Divide the array into two approximately equal halves.
2. **Conquer** – Recursively sort each half.
3. **Merge** – Combine the two sorted halves into one sorted array.

For example:

Original Array

↓

[8 3 5 2 7 1]

↓

[8 3 5] [2 7 1]

↓

[8] [3 5] [2] [7 1]

↓

Sorted smaller arrays

↓

[3 5 8] [1 2 7]

↓

[1 2 3 5 7 8]

---

3. Performance Testing

To analyze performance, Merge Sort can be executed using different input sizes such as:

Input Size (n)	Execution Time
1000	Measured time
5000	Measured time
10000	Measured time

Input Size (n)	Execution Time
20000	Measured time
50000	Measured time
100000	Measured time

The execution time can be measured using a programming language such as **C, C++, Java, or Python**.

The same experiment should be repeated for different types of input:

- Random data
- Already sorted data
- Reverse sorted data

The measured values can then be plotted using a graph.

---

4. Graphical Analysis

The graph should contain:

- **X-axis:** Input size n
- **Y-axis:** Execution time

Merge Sort Time Complexity Trend

Illustrative trend showing execution time increasing approximately as n log n.

inputSize	time
1,000	1

inputSize	time
5,000	7
10,000	15
20,000	32
50,000	85
100,000	175